VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI, KARNATAKA – 590018

A PROJECT REPORT ON

"DISTRIBUTED DENIAL OF SERVICE (DDoS) ATTACK DETECTION"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

NAME	USN
ADITI RUDRAKSHI	2BA22CS005
AMRUTA VIRAKTAMATH	2BA22CS013
ANANDKUMAR P. B.	2BA22CS014
BHARATH KUMAR K.	2BA22CS020

UNDER THE GUIDANCE OF

PROF SMITHA K.

ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B. V. V. SANGHA'S

BASAVESHWAR ENGINEERING COLLEGE, BAGALKOTE – 587102

2025 - 2026

B. V. V. SANGHA'S

BASAVESHWAR ENGINEERING COLLEGE, BAGALKOTE

Affiliated to

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI, KARNATAKA

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project work entitled **" DISTRIBUTED DENIAL OF SERVICE(DDoS) ATTACK DETECTION"** *is carried out by Ms. Aditi Rudrakshi, bearing USN: 2BA22CS005, Ms. Amruta Viraktamath, bearing USN: 2BA22CS013, Mr. Anandkumar Bhairamatti, bearing USN: 2BA22CS014 and Mr. Bharath Kumar Kharoshi, bearing USN: 2BA22CS020 in partial fulfilment of the requirements for the award of Bachelor of Engineering degree with specialization in Computer Science and Engineering is an authentic work carried out by them under our supervision and guidance.*

This Project satisfies the academic requirements prescribed for the said degree.

Guide	Coordinator	HOD	Principal
Prof. Smitha K.	**Prof. Netra Yaradoni**	**Dr. V. B. Pagi**	**Dr. B. R. Hiremath**

Name and Signature of Examiners

1. _____ _____

2. _____ _____

3. _____ _____

DECLARATION

We, **Aditi Rudrakshi (2BA22CS005), Amruta Viraktamath (2BA22CS013), Anandkumar P Bhairamatti (2BA22CS014), Bharath Kumar Kharoshi (2BA22CS020)** students of 7th semester B.E in Computer Science and Engineering, Basaveshwar Engineering College, Bagalkote, hereby declare that the project work entitled **"Distributed Denial of Service (DDoS) Attack Detection"** submitted to the Visvesvaraya Technological University during the academic year 2025-2026, is a record of an original work done by us under the guidance of **Prof Smitha K**, Assistant Professor, Department of Computer science and Engineering, Basaveshwar Engineering College, Bagalkote. This project work is submitted in fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

Date:

Place: Bagalkote

Aditi Rudrakshi

Amruta Viraktamath

Anandkumar P. B.

Bharath Kumar Kharoshi

ACKNOWLEDGEMENT

Completing a project is a significant milestone, and it would not have been possible without the support, guidance, and encouragement of those who contributed to its success. We take this opportunity to express our deepest gratitude to everyone who played a role in helping us achieve this accomplishment.

We are deeply grateful to **Shri B.V.V. Sangha's Basaveshwar Engineering College, Bagalkote**, for offering us the resources, facilities, and conducive environment that enabled us to accomplish our cherished goals.

We are deeply grateful to **Prof Smitha K**, our project coordinator and guide, for her unwavering support and invaluable guidance throughout the course of this project. Her extensive knowledge and insights provided us with the clarity and confidence to overcome challenges and stay focused on our objectives. We are sincerely thankful for her patience in addressing our doubts and for consistently motivating us to strive for excellence. Her ability to simplify complex concepts and encourage us to think critically has significantly enriched our learning experience, for which we will always remain thankful.

We owe a special debt of gratitude to our Head of the Department, **Dr. V. B. Pagi**, whose guidance and support played a pivotal role in the success of this project.

Our heartfelt thanks go to our beloved Principal, **Dr. B. R. Hiremath**, for his unwavering support and encouragement throughout our academic journey.

We also wish to acknowledge the teaching and non-teaching staff of the Department of Computer Science and Engineering for their timely cooperation, assistance, and encouragement, which greatly supported us in completing this project.

Finally, we thank our peers, family, and friends for their encouragement and support, which have been a constant source of motivation

CONTENTS

Chapter 1		Introduction	Page
1.1		Background	1
	1.1.1	Evolution of online services	2
	1.1.2	Rise of cyber threats in modern networks	2-3
	1.1.3	Emergence of distributed attacks	3-4
1.2		Understanding DDoS Attacks	4
	1.2.1	Definition and concept	5
	1.2.2	Types of DDoS Attacks	5
1.3		Challenges in detecting DDoS attacks	6-7
1.4		Problem Statement	8-9
1.5		Objectives	9
Chapter -2		**Literature survey**	**10-16**
Chapter -3		**System Requirements**	**17-23**
3.1		Hardware Requirements	17-19
3.2		Software Requirements	19-23
	3.2.1	Operating System	19
	3.2.2	Developmental Environment	19-20

	3.2.3	Libraries and Framework	20-23
3.3		Network Requirements	23
3.4		Dataset Requirements	23
3.5		Non- Functional Requirements	23
Chapter -4		**Proposed Methodology**	**24-26**
4.1		Overview of Proposed Methodology	24
4.2		System architecture	25-26
Chapter -5		**System Implementation**	**27-35**
5.1		Data collection	27
5.2		Data Preprocessing	27
5.3		Feature Engineering	28-30
5.4		Model Design and Training	31-35
Chapter -6		**Validation and Results**	**36-49**
6.1		Model Evaluation	36-38
6.2		Dashboard based validation	39-43
6.3		Live traffic capture testing	44-46
6.4		Local testing and trigger-based evaluation	47-49
		Conclusion	**50-51**
		References	**52-53**

List of Figures

Figure No.	Caption	Page No.
4.2.1	System Architecture	25
5.4.1	Feature Importance for XGBoost	33
6.1.1	Confusion matrix for machine learning model	37
6.1.2	Confusion matrix for deep learning model	38
6.1.3	Accuracy comparison	38
6.2.1	Dashboard Interface	40
6.2.2	Offline Testing Results Displayed on Dashboard for XGBoost	41
6.2.3	Offline Testing Results Displayed on Dashboard for XGBoost	41
6.2.4	Offline Testing Results Displayed on Dashboard for DNN	42
6.2.5	Offline Testing Results Displayed on Dashboard for DNN	43
6.3.1	Real Time Normal Traffic Detection Output using XGBoost model	45
6.3.2	Real Time DDoS Attack Detection Output using Deep Neural Network model	46
6.4.1	Trigger Based Detection Control for XGBoost Model	48
6.4.2	Trigger Based Detection Control for DNN Model	49

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

The rapid growth of digital technologies has transformed the way individuals, businesses, and governments operate. Modern society is increasingly dependent on interconnected platforms such as the Internet, cloud infrastructures, Internet of Things (IoT) networks, industrial automation systems, and large-scale online services. These ecosystems enable seamless communication, real-time data exchange, and global connectivity, forming the backbone of industries such as banking, healthcare, manufacturing, education, and e-commerce. However, this deep integration of digital systems has also widened the attack surface for cybercriminals. As connectivity expands, so does the potential for exploitation, making network security a critical concern in the digital era.

Within this landscape, cyberattacks have grown both in scale and sophistication. Among the most damaging forms of cyber threats are **Denial-of-Service (DoS) attacks** and **Distributed Denial-of-Service (DDoS) attacks**, which aim to disrupt the availability of digital services by overwhelming systems with excessive or malicious traffic. While DoS attacks are launched from a single source, their distributed variant—DDoS—is significantly more powerful, coordinated, and difficult to mitigate, as it originates from multiple compromised systems operating simultaneously. These attacks can halt essential services, cause significant downtime, inflict financial losses, and undermine user trust. As organizations continue to rely on digitally controlled operations, understanding the evolution and emergence of such attacks becomes essential for designing effective defence strategies.

1.1.1 Evolution of Internet Services

Over the past few decades, the Internet has evolved from a simple communication network into a highly complex global infrastructure supporting billions of devices and applications. The proliferation of cloud computing, virtualization, web services, big-data platforms, and IoT ecosystems has led to an exponential increase in the volume and variety of network traffic. Critical services—such as online banking, remote healthcare, e-governance, and industrial automation—now operate entirely on online platforms.

This evolution, while beneficial, has also created dependencies where continuous service availability is crucial. Any disruption can lead to operational breakdowns, financial setbacks, and safety risks. As services have grown more resource-intensive and widely distributed, attackers have exploited this dependency by launching traffic-based attacks such as DoS and DDoS. These attacks leverage the increased connectivity and bandwidth demands of modern systems, making them attractive methods for disabling or degrading service availability.

1.1.2 Rise of Cyber Threats in Modern Networks

The expansion of digital networks has been accompanied by a sharp rise in cyber threats. Modern adversaries deploy sophisticated attack strategies that take advantage of weaknesses in communication protocols, system vulnerabilities, and insufficient security measures. Threat actors range from individual hackers to organized cybercriminal groups, hacktivists, and state-sponsored entities.

Among these threats, **Denial-of-Service (DoS) attacks** have long been used to disrupt servers by flooding them with illegitimate traffic. A traditional DoS attack originates from a single machine sending repeated requests or packets to exhaust server resources such as CPU cycles, memory, or bandwidth. While impactful, such attacks can often be detected and mitigated due to their single-source nature.

In contrast, the rise of **Distributed Denial-of-Service (DDoS) attacks** has marked a significant escalation. DDoS attacks utilize vast networks of infected devices—commonly referred to as botnets—to generate large-scale, coordinated traffic floods. These devices include compromised

computers, IoT gadgets, routers, and even smart appliances. By distributing malicious traffic across hundreds or thousands of devices, attackers make detection and filtering far more difficult. Additionally, modern DDoS attacks can mimic legitimate user behavior, making them harder to distinguish from normal traffic, thereby increasing their severity and frequency.

1.1.3 Emergence of Distributed Attacks

The transition from single-source DoS attacks to large-scale DDoS attacks has been driven by advancements in malware, the growth of insecure IoT devices, and the increasing availability of high-bandwidth networks. Attackers today leverage automated tools to infect vulnerable systems and convert them into remotely controlled bots. These botnets execute coordinated DDoS attacks aimed at overwhelming target servers, network infrastructures, or online services. Unlike DoS attacks, which rely on the capacity of a single attacker, distributed attacks harness collective power from many compromised sources, allowing them to generate massive traffic volumes. This distribution makes it extremely challenging for defenders to identify attack sources, trace origins, or block malicious traffic without affecting legitimate users. DDoS attacks can target different layers of the network stack, including volumetric floods that saturate bandwidth, protocol attacks that exploit weaknesses in network communication, and application-layer attacks that mimic normal user requests while overloading server resources. The emergence of these distributed threats has made DDoS one of the most disruptive and persistent cybersecurity challenges in modern networks. As a result, the cybersecurity community increasingly focuses on intelligent and adaptive detection methods, particularly those utilizing machine learning and deep learning, to identify abnormal traffic patterns in real time.

During 2025, the Indian cyber threat landscape witnessed a notable escalation in large-scale cyberattacks coinciding with heightened geopolitical tensions following *Operation Sindoor*, a military campaign undertaken by India in response to a major terror incident. As this operation unfolded, Indian digital infrastructure—including government portals, financial services, power systems, and critical industrial networks—became the target of extensive malicious activity, prominently including Distributed Denial-of-Service (DDoS) attacks launched by state and non-state actors. According to official and investigative reports, hundreds of thousands of

cyberattacks were registered against Indian systems, with adversaries attempting to overwhelm online services with high volumes of illegitimate traffic to disrupt availability and challenge national defence readiness. In one widely reported instance, a group of attackers allegedly coordinated DDoS attacks against more than 50 Indian government websites via Telegram-based channels, leading to arrests by the Gujarat Anti-Terrorism Squad under cybercrime provisions of Indian law.

1.2 Understanding DDoS Attacks

Distributed Denial-of-Service (DDoS) attacks represent one of the most disruptive and widely deployed forms of cyber aggression in modern networked environments. Their primary objective is to render a target system, service, or network unavailable by overwhelming it with excessive, malicious, or strategically crafted traffic. Unlike conventional cyberattacks that aim to steal data or infiltrate systems, DDoS attacks focus specifically on degrading service availability—the very foundation of online operations. As organizations increasingly depend on uninterrupted digital services, DDoS attacks have evolved into high-impact threats capable of crippling critical infrastructure, online platforms, and business operations. A DDoS attack is fundamentally an extension of the simpler Denial-of-Service (DoS) attack. However, while a DoS attack originates from a single machine or a limited set of sources, a DDoS attack leverages a large distributed network of compromised devices—commonly known as botnets—to carry out coordinated assaults. These botnets are often composed of infected personal computers, servers, and increasingly IoT devices, which are typically easy to compromise due to weak security configurations. By orchestrating thousands of such devices simultaneously, attackers generate massive volumes of traffic that exceed the target's handling capacity, thereby causing service degradation or complete outage.

1.2.1 Definition and Basic Concept

At its core, a DDoS attack is an intentional effort to exhaust the computing, memory, or bandwidth resources of a target system. Attackers send a high number of requests, packets, or malformed traffic to overwhelm the system's processing capabilities. When the target system becomes overloaded, legitimate users can no longer access the service, resulting in downtime, operational disruption, and potential financial loss. The distributed nature of DDoS attacks makes detection highly challenging because malicious traffic is dispersed across many globally located devices, making it difficult to differentiate from normal traffic patterns.

1.2.2 Types of DDoS Attacks

DDoS attacks can occur at multiple layers of the network and are typically categorized into three major types:

a. Volumetric Attacks

These attacks aim to saturate the target's bandwidth by generating extremely high volumes of traffic. Examples include UDP floods and amplification attacks such as DNS or NTP amplification. Such attacks attempt to consume all available network capacity, leaving no room for legitimate communication.

b. Protocol Attacks

Protocol-based attacks exploit vulnerabilities in network protocols or how systems manage connections. Common examples include SYN floods, Smurf attacks, and fragmented packet attacks. These attacks target the resources of servers, load balancers, or firewalls, causing them to malfunction or crash.

c. Application-Layer Attacks

These attacks mimic legitimate user requests but in large volumes, targeting specific applications such as HTTP, DNS, or SSL services. Examples include HTTP GET/POST floods and Slowloris attacks. Since the requests appear legitimate, application-layer attacks are often much harder to detect and mitigate.

1.3 Challenges in Detecting DDoS Attacks

Detecting Distributed Denial of Service (DDoS) attacks presents numerous technical and operational complexities, primarily because these attacks continue to evolve in scale, sophistication, and execution techniques. Modern digital infrastructures—cloud platforms, critical government portals, e-commerce applications, and IoT-enabled networks—are exposed to increasingly stealthy adversaries who leverage distributed systems, botnets, and intelligent evasion strategies. As a result, identifying DDoS traffic in real time has become a significant research challenge.

1. High Traffic Volumes and Amplification

Attackers often generate traffic volumes that resemble legitimate surges such as flash crowds, seasonal spikes, or software update distributions. Volumetric DDoS attacks exploit bandwidth and amplification protocols (e.g., DNS amplification, NTP amplification) to overwhelm network capacity. The sheer magnitude makes it difficult to differentiate between genuine high-load scenarios and malicious floods without advanced detection mechanisms.

2. Distributed Attack Sources and Botnets

Most DDoS attacks originate from globally distributed botnets consisting of compromised systems, IoT devices, or cloud instances. Since traffic comes from thousands or millions of geographically dispersed sources, identifying malicious IPs or traffic signatures becomes non-trivial. Attackers also frequently rotate IP addresses, spoof identities, and use legitimate-looking request patterns to bypass basic filtering.

3. Protocol Complexity and Multi-Vector Attacks

Modern attackers combine multiple attack vectors—volumetric, protocol-based, and application-layer attacks—in a coordinated manner. Multi-vector strategies make traditional threshold-based detection insufficient because each vector behaves differently. For example, SYN floods may target network layers, while HTTP GET/POST floods attack the application layer, requiring multi-level inspection and correlation.

4. Encrypted Traffic and Reduced Visibility

With the widespread adoption of HTTPS and TLS, a large portion of network traffic is encrypted. Although encryption improves end-user security, it limits visibility for intrusion detection

systems because payload inspection becomes computationally expensive or infeasible. Attackers exploit this by launching encrypted DDoS attacks that blend seamlessly with legitimate secure sessions.

5. Low-Rate and Stealthy DDoS Attacks

While traditional DDoS attacks relied on high-volume floods, attackers now increasingly use low-rate, slow-and-silent techniques. These attacks generate minimal traffic but exploit protocol timeouts and connection limits. Examples include Slowloris, Slow POST, and low-bandwidth congestion attacks. Their subtle patterns often mimic normal user behaviour, making anomaly detection highly challenging.

6. Dynamic Network Behaviour and Real-Time Requirements

Modern networks are dynamic due to auto-scaling, virtualization, containerization, and SDN-based configurations. Normal traffic patterns change constantly, reducing the accuracy of static threshold-based systems. Effective DDoS detection requires real-time processing, adaptive baselines, and predictive analytics—further increasing computational overhead.

7. Adversarial Evasion Techniques

Attackers increasingly employ evasion techniques such as:

- Traffic randomization
- Protocol spoofing
- Burst-based attacks
- Use of legitimate cloud services for masking

These techniques help attackers blend malicious traffic with benign flows, often bypassing rule-based and signature-based systems. Consequently, detection models must incorporate advanced behavioural analytics and machine learning to counter evolving evasion strategies.

8. Limitations of Traditional Security Systems

Most legacy systems were designed for known threats and predefined rules rather than dynamic, large-scale distributed attacks. This gap has led to a growing interest in AI-based, data-driven detection frameworks capable of recognizing anomalies within high-dimensional traffic features.

1.4 Problem Statement

The rapid proliferation of digital services, cloud computing frameworks, IoT deployments, and large-scale online platforms has significantly expanded the attack surface of modern networks. Among the various cyber threats targeting these interconnected environments, Distributed Denial of Service (DDoS) attacks remain one of the most persistent and damaging. These attacks aim to exhaust critical resources—such as bandwidth, processing capacity, and application-level services—thereby rendering legitimate users unable to access essential systems. Despite significant advancements in network security tools, detecting DDoS attacks accurately and in real time continues to be a major challenge for enterprises, governments, and service providers.

Traditional security mechanisms such as firewalls, threshold-based alerts, or signature-driven intrusion detection systems are insufficient against today's sophisticated and multi-vector DDoS attacks. Modern attackers often leverage globally distributed botnets, cloud-hosted malware, and advanced evasion techniques that closely mimic legitimate traffic patterns. As a result, distinguishing abnormal traffic flows from genuine user activities becomes increasingly complex. In addition, encrypted communication, dynamic traffic behaviour, and low-rate stealth attacks further reduce visibility and degrade the performance of conventional detection techniques.

This gap between evolving attack patterns and outdated detection mechanisms creates a critical need for intelligent, adaptive, and data-driven approaches capable of analyzing high-dimensional traffic features. Machine learning and deep learning models offer promising solutions; however, the challenge lies in designing systems that can generalize to diverse network environments, handle imbalanced datasets, and operate effectively without relying on raw packet captures—especially in operational environments where real-time packet sniffing may not be feasible.

Therefore, the core problem addressed by this study is the development of an efficient, scalable, and accurate DDoS detection framework that can analyse extracted traffic features, differentiate between legitimate and malicious flows, and deliver reliable predictions under dynamic and high-volume network conditions. The study also aims to overcome issues such as feature redundancy, dataset imbalance, limited visibility in encrypted networks, and the increasing sophistication of

attack strategies. The problem requires a holistic methodology that combines robust feature engineering, effective ML/DL modelling, and performance evaluation capable of supporting real-world deployment in security-critical environments.

1.5 Objectives

- To design and develop an intelligent DDoS attack detection system using machine learning and deep learning techniques that can accurately distinguish malicious traffic from legitimate network traffic by analyzing meaningful network features.
- To train, evaluate, and compare XGBoost and Deep Neural Network (DNN) models using standard performance metrics such as accuracy, precision, recall, and F1-score, in order to identify the strengths and effectiveness of each approach.
- To implement a real-time, scalable detection framework with visualization support, enabling efficient live traffic monitoring, clear interpretation of model predictions, and practical deployment in real-world network security environments.

CHAPTER 2

LITERATURE REVIEW

1. S. Abhiramsundari and V. Ramaswamy, "Distributed denial-of-service (DDoS) attack detection using supervised learning algorithms", Scientific Reports, 2025

This study examines the application of supervised machine learning techniques for detecting Distributed Denial-of-Service (DDoS) attacks and highlights the challenges associated with differentiating legitimate traffic from botnet-driven attack traffic. The authors utilize multiple datasets to evaluate the robustness of learning models and implement preprocessing methods such as oversampling, dimensionality reduction, and label encoding to handle dataset imbalance, reduce noise, and enhance feature representation. The research underscores the complexity of detecting botnet-based DDoS attacks due to their ability to mimic normal network behavior. It also emphasises the relevance of ensemble learning approaches, which integrate the strengths of multiple algorithms for improved detection reliability. Additionally, the paper recognises the significance of understanding attacker motivations, as this knowledge aids in designing effective prevention and mitigation strategies in evolving threat environments. Additionally, the study's discussion on dimensionality reduction reinforces the importance of eliminating redundant or noisy features that can mislead classifiers and increase computational overhead. The paper further underscores the value of ensemble learning approaches, which combine multiple algorithms to improve detection reliability and resilience against diverse attack strategies. This directly supports project designs that integrate multiple classifiers or leverage ensemble-based models for higher robustness.

2. Xingbing Fu and Supeng Lou, "Deep Learning Techniques for DDoS Attack Detection", 2025

This paper presents a detailed discussion on the role of deep learning models in detecting DDoS attacks, focusing on their capability to learn complex, non-linear patterns from network traffic. The authors argue that conventional signature-based detection systems are limited because they rely on known attack patterns and may not detect new or modified attack variants. In contrast, deep

learning models automatically extract high-level representations from large volumes of both benign and malicious traffic, enabling them to identify subtle deviations indicative of abnormal behaviour. The study highlights how neural networks can learn temporal correlations, traffic dependencies, and intricate feature relationships without extensive manual feature engineering. The paper also emphasizes that neural networks can capture temporal correlations and inter-packet dependencies, making them suitable for detecting slow-rate, low-and-slow, or multi-stage DDoS attacks. This aligns well with projects that incorporate time-windowed features or sequential models to improve detection accuracy in dynamic traffic environments.

3. Raihan Putra Janivasya and Ika Dyah Agustia Rachmawati, "DDoS Detection Using Machine Learning Approach," Procedia Computer Science, 2024

This research investigates the mechanisms of DDoS attacks and examines machine learning approaches for detecting abnormal network traffic. The study explores the UNSW-NB15 dataset, which contains a wide range of modern network behaviours, and evaluates several learning algorithms spanning linear models, distance-based models, kernel-based methods, tree-based classifiers, and recurrent neural network architectures. The paper adopts a structured methodology consisting of feature extraction, data preprocessing, algorithm training, and model evaluation using standard classification metrics. The authors emphasize the relevance of analysing network flow attributes to identify traffic anomalies that signify potential DDoS attacks. Their work highlights the growing role of machine learning in developing systematic, automated, and intelligent network defence strategies capable of adapting to varied attack behaviours. Furthermore, the comparative evaluation across multiple algorithm families provides valuable insights into how different learning paradigms perform under the same dataset conditions. This enables projects to justify their algorithm selection based on empirical evidence rather than convenience. The paper also reinforces the importance of standard evaluation metrics, ensuring that project results are comparable with existing literature.

4. Haya Malooh Alqahtani and Monir Abdullah, "A Review on DDoS Attacks Classifying and Detection by ML/DL Models," IJACSA, 2024

This review paper surveys existing research on DDoS detection using machine learning and deep learning models. It outlines the increasing prevalence of DDoS attacks and the corresponding need for accurate classification strategies for securing network infrastructures. The study categorises prior works based on algorithm families, commonly used datasets, and methodological trends observed in the literature. It notes the widespread use of both classical ML techniques and advanced DL architectures, reflecting a shift toward automated feature extraction and representation learning. The authors observe that researchers often rely on custom datasets or enhanced versions of existing datasets to address specific attack scenarios, while benchmark datasets such as NSL-KDD remain widely adopted for comparative studies. The review identifies prominent themes, methodological gaps, and future research opportunities related to dataset diversity, model generalisation, and scalable deployment techniques. The review also highlights persistent challenges such as dataset diversity, lack of real-world traffic realism, and limited generalization across environments. These identified gaps can be directly addressed in projects by combining datasets, validating models on unseen traffic, or designing hybrid detection frameworks. The paper thus helps justify project objectives aimed at improving robustness and scalability.

5. Mohammad Najafimehr, Sajjad Zarifzadeh, and Seyed Akbar Mostafavi, "DDoS Attacks and Machine-Learning-Based Detection Methods: A Survey and Taxonomy," Engineering Reports, 2023

This survey presents a structured taxonomy of machine-learning-based DDoS detection methods and analyses the challenges associated with securing networks against evolving attack strategies. The paper classifies learning approaches into supervised, unsupervised, and hybrid categories and discusses their respective strengths for anomaly detection and signature identification. It provides an extensive review of datasets commonly used in DDoS research and explains their limitations, including issues related to class imbalance, outdated traffic patterns, and the need for realistic representations of modern network environments. Additionally, the authors explore emerging

challenges such as high-speed traffic processing, encrypted traffic inspection, and adaptive attack behaviours. The survey aims to support researchers and practitioners by offering a comprehensive reference for developing more robust and scalable DDoS detection frameworks.

6. Mahmood A. Al-Shareeda, Selvakumar Manickam, and Murtaja Ali Saare, "DDoS Attacks Detection Using Machine Learning and Deep Learning: Analysis and Comparison", 2023

This paper examines both machine learning and deep learning techniques in the context of DDoS attack detection and provides insights into when each class of methods is most appropriate based on data characteristics and modelling requirements. The authors outline fundamental differences between ML and DL approaches, noting that ML models typically require structured data, feature engineering, and manual tuning, making them suitable for scenarios involving limited datasets and rapid training needs. Conversely, the study explains that deep learning techniques reduce reliance on manual feature extraction by learning hierarchical patterns directly from raw or minimally processed data, making them advantageous for large-scale datasets and complex traffic structures. The paper strengthens the argument that DL-based solutions are better suited for modern, high-volume network environments, while ML approaches remain relevant for lightweight or resource-constrained systems.

7. Sharmin Aktar and Abdullah Yasin Nur, "Towards DDoS Attack Detection Using Deep Learning Approach", 2023

This work proposes a deep learning approach for DDoS detection based on a contractive autoencoder, which functions within a semi-supervised learning framework. The study trains the model exclusively on normal traffic, enabling it to learn compact representations and reconstruct benign input with minimal error. The authors describe how anomalies—representing attack traffic—produce significantly higher reconstruction errors because the model lacks familiarity with their behavioural patterns. By leveraging reconstruction error as a decision boundary, the system differentiates between normal and anomalous traffic without requiring labelled attack samples during training. The paper underscores the relevance of reconstruction-based anomaly detection

techniques for environments where acquiring diverse and complete attack datasets is difficult. This approach is highly suitable for real-world environments where collecting comprehensive attack datasets is impractical. The paper supports project designs that prioritise **anomaly-based detection**, especially for identifying previously unseen or evolving DDoS attacks.

8. Sheeraz Ahmed et al., "Effective and Efficient DDoS Attack Detection Using Deep Learning Algorithm, Multi-Layer Perceptron," Future Internet, 2023

This study concentrates on identifying application-layer DDoS attacks, particularly those designed to resemble legitimate user behaviour, which makes them difficult to detect using conventional traffic inspection techniques. The authors analyse detailed packet-level attributes such as HTTP payload size, source–destination address patterns, port distributions, and proxy utilisation to characterise subtle differences in browsing and request behaviours. The work integrates heterogeneous datasets, including real weblogs, CTU-13 data, and traffic generated by application-layer attack tools, to construct a diverse training corpus. A multilayer perceptron architecture is adopted to learn behavioural distinctions and identify malicious sessions. The paper emphasises the value of deep learning for recognising high-level traffic signatures and latent behavioural patterns that cannot be captured through simple rule-based methods.

9. Meenakshi Mittal, Krishan Kumar, and Sunny Behal, "Deep Learning Approaches for Detecting DDoS Attacks: A Systematic Review," Artificial Intelligence, 2022

This systematic review analyses the growing body of literature on deep learning applications for DDoS detection. It examines the evolution of attack techniques in parallel with increases in global internet traffic and highlights why traditional, rule-based detection systems struggle to adapt to new threats. The authors review studies from leading academic sources, categorising them by neural network architectures, feature extraction strategies, datasets, and evaluation methodologies. The paper identifies common experimental practices, recurring research gaps, and limitations related to dataset authenticity, data preprocessing, and model interpretability. It concludes by emphasising the need for more realistic datasets, hybrid detection architectures, and models capable of handling dynamic network conditions in real-time operational environments. The paper highlights issues such as dataset authenticity, preprocessing inconsistencies, and lack of interpretability, which are important considerations for project design. By acknowledging these limitations, projects can incorporate explainability, cross-dataset validation, or hybrid models to improve trust and usability.

10. Firooz B. Saghezchi, Georgios Mantas, Manuel A. Violas, "Machine Learning for DDoS Attack Detection in Industry 4.0 CPPSs," 2022

This research investigates the use of machine learning models for detecting DDoS attacks in Industry 4.0 environments, particularly Cyber-Physical Production Systems (CPPSs), where interconnected sensors, devices, and industrial machinery generate continuous high-volume traffic. The authors analyse network traffic traces extracted from a large-scale semiconductor factory and evaluate multiple learning approaches across supervised, semi-supervised, and unsupervised categories. The study highlights practical challenges associated with industrial networks, including heterogeneous communication patterns, timing sensitivities, and the need for detection methods that can operate reliably under real-world production conditions. The findings discuss the behaviour of various modelling techniques before and after dimensionality reduction using PCA, offering insights into their suitability for anomaly detection within complex industrial

infrastructures. The analysis of model performance before and after dimensionality reduction provides insights into balancing detection accuracy and computational efficiency. This is particularly relevant for projects aiming to deploy detection systems in real-time or resource-constrained environments.

Overall, the study supports the development of scalable and adaptable DDoS detection frameworks capable of operating under real-world industrial conditions.

CHAPTER 3

SYSTEM REQUIREMENTS

The proposed DDoS detection system is designed to operate on commodity computing hardware, ensuring accessibility while maintaining sufficient computational capacity for machine learning model execution and real-time network traffic analysis. Hardware requirements are influenced by factors such as dataset size, feature extraction complexity, model architecture, and traffic monitoring rate. The system can be deployed on standalone workstations or integrated into server environments depending on operational needs.

The minimum and recommended hardware specifications are outlined to ensure reliable execution of preprocessing pipelines, inference modules, and continuous packet or flow monitoring without significant latency.

3.1 Hardware Requirements

The minimum hardware configuration supports offline training, model testing, and low-to-moderate traffic analysis. This configuration is suitable for academic experimentation, proof-of-concept implementations, and small-scale deployments.

- Processor (CPU): Intel Core i3 / AMD Ryzen 3 or equivalent
  A dual-core or quad-core processor is sufficient to execute basic feature extraction, scaling, and inference tasks for classical ML models and lightweight deep learning architectures.

- Clock Speed: 2.0 GHz or higher
  Adequate clock speed ensures timely processing of network features and prevents bottlenecks during real-time prediction.

- RAM: 4 GB
  Supports loading trained models, preprocessing modules, and handling moderate

volumes of network traffic data. Suitable for batch-based evaluation and controlled live testing.

- Storage: 50 GB free disk space (HDD/SSD)
  Required for storing datasets, trained model files, logs, and intermediate feature representations generated during traffic analysis.

- Network Interface: Ethernet/Wi-Fi adapter
  Enables packet capture or flow monitoring for live traffic analysis. A stable network interface is essential for accurate data acquisition.

- Display: 1366 × 768 resolution monitor
  Provides adequate resolution for monitoring system output, logs, and performance metrics during experimentation and debugging.

Recommended Hardware Requirements

The recommended configuration is intended for real-time deployment, high-volume traffic monitoring, and deep learning-based inference. This setup enhances system responsiveness, scalability, and detection accuracy under realistic network loads.

- **Processor (CPU):** Intel Core i5 / AMD Ryzen 5 or equivalent
  multi-core processors significantly improve parallel execution of traffic capture, feature computation, and model inference, reducing detection latency.

- **Clock Speed:** 2.5 GHz or higher
  Higher clock speeds support faster execution of preprocessing pipelines and improve real-time responsiveness.

- **RAM:** 8 GB or more
  Enables efficient handling of larger traffic windows, multiple concurrent processes, and complex DL models without memory constraints.

- **Storage:** 100 GB free SSD
  Solid-state storage improves data read/write speeds, which is beneficial for

logging high-frequency traffic data, loading models, and maintaining historical records for analysis.

- **GPU (Optional):** NVIDIA GPU with CUDA support
  GPU acceleration significantly reduces inference time for deep learning models such as DNNs, CNNs, or LSTM-based architectures. While not mandatory, GPU support enhances performance for large-scale or continuous real-time detection environments.

3.2 Software Requirements

The software stack is designed to support data preprocessing, machine learning model execution, and web-based visualization of results.

3.2.1 Operating System

The proposed DDoS attack detection system can be developed and executed across multiple operating systems to ensure flexibility, portability, and ease of deployment. Windows 10/11 is suitable for general development tasks due to its wide compatibility with Python tools and IDEs. Ubuntu 20.04 LTS or later is recommended for production or research environments requiring high stability, better resource control, and native support for machine learning frameworks and GPU acceleration. macOS 11 or later offers an optimized Unix-based environment with strong developer tools and seamless integration for Python-based workflows. Supporting these operating systems ensures that the system remains accessible to researchers, developers, and deployment teams across diverse platforms.

3.2.2 Development Environment

The proposed DDoS attack detection system is developed using a robust and flexible software environment designed to support data preprocessing, model training, visualization, and deployment. The primary development platform is Python 3.8 or higher, chosen due to its extensive ecosystem of scientific and machine learning libraries, ease of integration, and strong community support. All

model prototyping, experimentation, and evaluation were carried out using Jupyter Notebook, which provides an interactive computational environment suitable for iterative testing and visualization. For modular development and production-level implementation, VS Code and PyCharm were employed. These Integrated Development Environments (IDEs) offer advanced debugging, environment management, version control integration, and code refactoring capabilities, which collectively enhance software quality, maintainability, and developer productivity.

3.2.3 Libraries and Frameworks

A wide range of specialized libraries and frameworks were utilized to support various components of the proposed DDoS detection pipeline. Each tool contributes unique capabilities essential for data handling, model development, training optimization, and user interface deployment. The following are the key libraries and their roles:

1. **NumPy – Numerical Computations**
   NumPy serves as the foundational library for performing high-performance numerical operations. It provides optimized multidimensional arrays, vectorized operations, linear algebra routines, and mathematical functions that significantly accelerate data transformations and feature engineering tasks. Its efficiency is critical when handling large-scale network traffic datasets.

2. **Pandas – Dataset Handling and Preprocessing**
   Pandas is employed for structured dataset manipulation, cleaning, and preprocessing. Its DataFrame structure allows efficient handling of tabular network flow records, enabling operations such as filtering, grouping, feature extraction, time-based aggregation, and missing value management. Pandas also facilitates seamless integration with NumPy and Scikit-learn pipelines.

3. **Scikit-learn – Machine Learning Algorithms**
   Scikit-learn provides an extensive suite of classical machine learning algorithms, including Decision Trees, Random Forest, k-Nearest Neighbors (kNN), and Support Vector Machines (SVM), which are central to baseline model development. Additionally, it offers tools for data scaling, model evaluation, hyperparameter tuning, and cross-validation, enabling rigorous experimentation and comparison.

4. **TensorFlow / PyTorch – Deep Learning Model Implementation**
   TensorFlow and PyTorch are state-of-the-art deep learning frameworks used for implementing advanced neural network architectures. Their modular APIs facilitate the design and training of multi-layer DNNs, LSTMs, and hybrid models. Both frameworks provide GPU acceleration, automatic differentiation, and extensive model optimization utilities, making them essential for handling complex temporal and spatial patterns in network traffic.

5. **XGBoost – Gradient Boosting Models**
   XGBoost is utilized for building high-performance gradient boosting models, known for their ability to handle nonlinear feature interactions and deliver superior predictive accuracy. Its regularization mechanisms, built-in cross-validation, and parallelized tree boosting make it particularly effective for detecting subtle anomalies indicative of DDoS attacks.

6. **Streamlit – Web-based User Interface**
   Streamlit is incorporated for rapid development of the system's web-based inference interface. It enables deployment of machine learning and deep learning models with minimal overhead, providing interactive components for user input, model selection, and real-time prediction visualization. Its simplicity and seamless integration with Python make it well suited for research prototypes and demonstration dashboards.

7. **Matplotlib / Seaborn – Data Visualization**
   Matplotlib and Seaborn are used to generate detailed visualizations that support exploratory data analysis (EDA) and model evaluation. Matplotlib offers low-level control for constructing custom plots, while Seaborn provides aesthetically refined statistical visualizations. These libraries assist in understanding traffic distributions, feature correlations, anomaly patterns, and performance metrics through graphs, heatmaps, and trend charts.

8. **Joblib / Pickle – Model and Scaler Persistence**

   Joblib and Pickle are Python serialization libraries used to store trained machine learning models and preprocessing objects such as feature scalers in a persistent format. These libraries enable the system to save learned parameters after the training phase and reload them during real-time prediction without retraining. Joblib is particularly optimized for large numerical objects and NumPy arrays, making it well-suited for Scikit-learn and deep learning models. Pickle, being Python's native serialization module, provides flexibility in storing general Python objects and configurations. In this project, trained ML and DNN models along with their corresponding scalers are saved using these libraries. During live traffic analysis, the saved models are loaded to ensure consistent and accurate predictions. This approach reduces computational overhead and improves system efficiency. Model persistence also guarantees feature alignment between training and inference phases. Overall, Joblib and Pickle play a critical role in maintaining reliability and reproducibility of the detection system.

9. **Scapy– Packet Sniffing and Traffic Capture**

   Scapy are networking libraries used for capturing and analyzing live network traffic in real time. Scapy provides high-level packet manipulation capabilities, allowing inspection of packet headers, protocol types, port numbers, and packet sizes. It is widely used in network security research due to its flexibility and protocol-level control.The captured packets are forwarded for feature extraction and classification. Reliable packet sniffing is essential for accurate detection of abnormal traffic patterns. Thus, Scapy form the foundation of real-time DDoS traffic monitoring.

10. **Socket and Threading – Concurrent Packet Processing**

    The socket and threading modules are used to enable efficient and concurrent processing of network traffic. The socket module provides low-level interfaces for network communication, allowing the system to receive and manage continuous streams of packet data. Threading allows multiple tasks to execute simultaneously within a single application. In this project, packet capture, feature extraction, and

model prediction are handled in separate threads. This design prevents blocking and ensures uninterrupted packet sniffing during inference. Threading significantly reduces latency in real-time detection scenarios. It also improves system responsiveness under high traffic conditions. By enabling parallel execution, the system can scale to handle larger traffic volumes. Together enhance the performance and robustness of the live detection framework.

3.3 Network Requirements

The system requires a stable network environment to collect and analyze traffic data.

- Internet access for detection
- Localhost traffic support for testing

3.4 Dataset Requirements

The system uses benchmark datasets and real-time captured traffic for training and testing.

- Public DDoS datasets such as:
    - CIC-DDoS2017
- Pre-processed CSV feature-based datasets for model input

3.5 Non-Functional Requirements

These requirements define system qualities and operational constraints:

- Performance: Attack detection latency should be less than 2 seconds per request.
- Scalability: System should handle high traffic volume and large datasets.
- Security: Input data must be validated to prevent injection or tampering.
- Reliability: System uptime should be above 95%.
- Usability: User-friendly web interface with real-time alerts.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Overview of Proposed Methodology

The proposed methodology focuses on designing an intelligent and efficient system for detecting Distributed Denial of Service (DDoS) attacks using machine learning and deep learning techniques. The system aims to analyze network traffic patterns and classify them as either normal or attack traffic in real time. Unlike traditional rule-based intrusion detection systems, the proposed approach leverages data-driven models that can learn complex traffic behaviors and adapt to evolving attack patterns.

The methodology begins with the collection of network traffic data containing both normal and DDoS attack instances. From this data, relevant network features such as packet rate, flow duration, packet size statistics, protocol type, and port information are extracted. These features play a crucial role in differentiating legitimate traffic from malicious traffic. The extracted data is then preprocessed by handling missing values, selecting important features, and applying normalization techniques to improve model performance.

The processed data is used to train two types of models: a machine learning model using XGBoost and a deep learning model using a Deep Neural Network (DNN). The XGBoost model is used for its fast execution and high accuracy on structured data, while the DNN model is used to capture complex and non-linear relationships in network traffic. Both models are trained offline and evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score.

Once trained, the models are deployed for real-time detection. Live network traffic is continuously monitored, features are extracted in real time, and the trained models classify the traffic as normal or attack. A trigger-based mechanism is incorporated to control live monitoring and improve system efficiency. The final output is displayed through a simple visualization interface, allowing users to easily understand the network status. This hybrid methodology ensures accurate, scalable, and real-time DDoS attack detection.

4.2 System Architecture

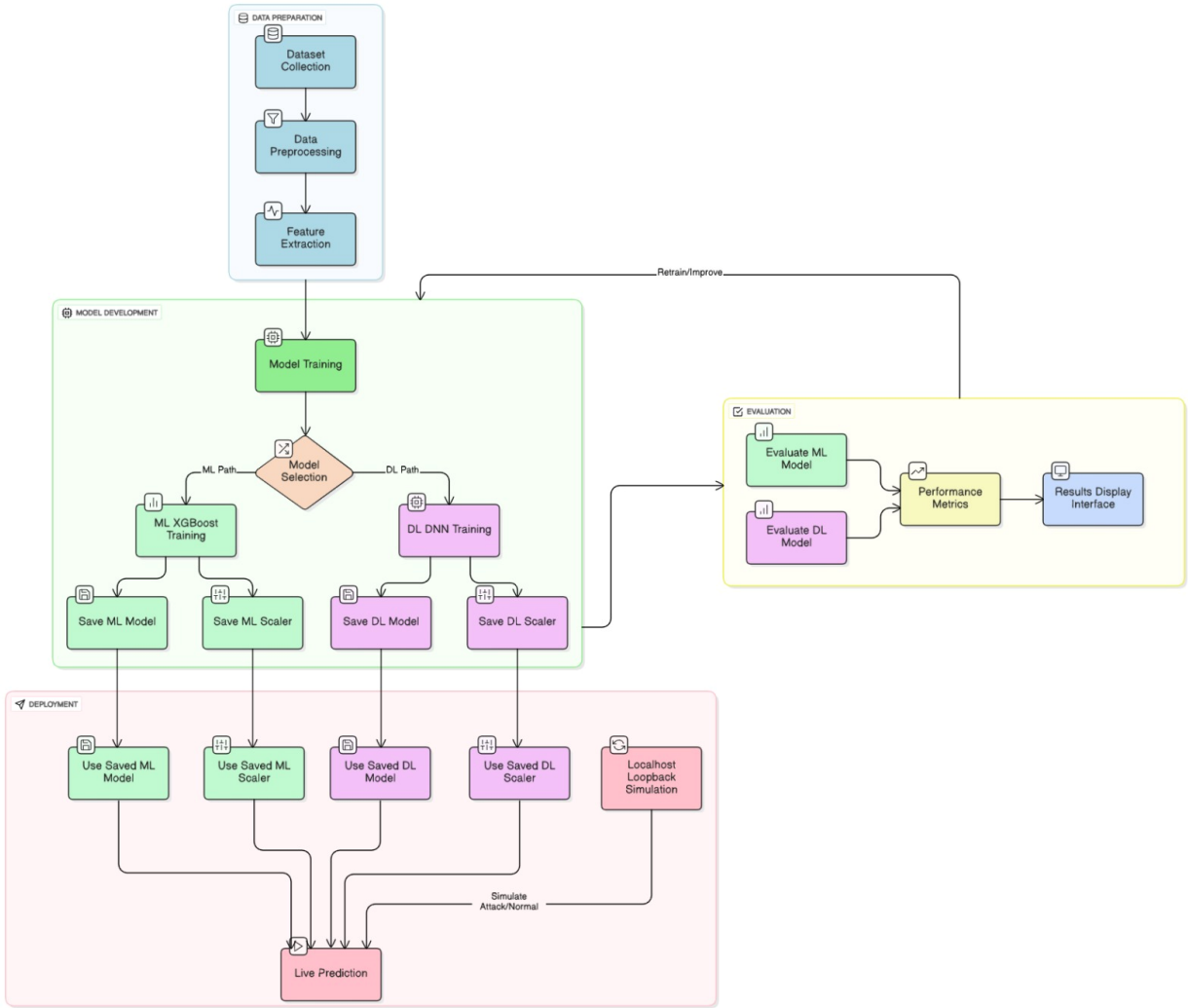


Fig 4.2.1 System Architecture

The first component of the architecture is the **Data Collection Module**, which gathers network traffic data from standard datasets during the training phase and from live network interfaces during deployment. This module ensures the availability of both normal and attack traffic samples required for building and validating the detection models.

The collected data is passed to the **Preprocessing and Feature Extraction Module**, where raw network traffic is cleaned, normalized, and transformed into meaningful features such as

packet rate, flow duration, packet size statistics, protocol type, and port information. These extracted features are formatted to match the input requirements of the machine learning and deep learning models.

Next, the **Model Training Module** is responsible for training the detection models using the preprocessed dataset. This module consists of two sub-components: a **Machine Learning module using XGBoost** and a **Deep Learning module using a Deep Neural Network (DNN)**. Both models are trained offline and evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. After successful training, the models and preprocessing objects are saved for future use.

Following model training, a **Visualization and Performance Comparison Module** is introduced. This module analyzes the evaluation results of both XGBoost and DNN models and presents their performance through visual representations such as graphs and charts. It helps in comparing detection accuracy, precision, recall, and F1-score of the models. This visualization enables better understanding of model behavior and assists in identifying the most effective model for real-time deployment.

The **Live Traffic Monitoring Module** captures real-time network packets from the system's network interface and extracts features in the same format used during training. These features are then forwarded to the **Prediction Module**, where the trained XGBoost and DNN models classify the traffic as either normal or DDoS attack.

A **Trigger Control Module** is incorporated to manage the start and stop of live detection. This module improves system efficiency by allowing controlled execution of real-time monitoring and reducing unnecessary computation during idle periods.

Finally, the **Result Display Module** presents the real-time prediction results to the user in a clear. It displays the network status as normal or under attack, enabling quick interpretation and timely response. Overall, this architecture ensures seamless integration of data collection, model training, visualization, real-time detection, and result presentation, making the system suitable for practical network security applications.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Data Collection

The first step involves collecting both normal and malicious network traffic data from reliable sources. The dataset consists of labelled network traffic samples that represent various types of DDoS attacks.

Data Sources:

- Standard benchmark datasets such as CICIDS2017 by Ameur Salem

5.2 Data Preprocessing

Raw network traffic data is noisy and inconsistent. Therefore, preprocessing is performed to improve data quality.

Key preprocessing steps:

- Removal of duplicate and corrupted records
- Handling of missing values using statistical imputation
- Label encoding of categorical attributes
- Normalization and standardization of numerical features using Min-Max Scaling or Z-score normalization
- Balancing of data using oversampling (SMOTE) or undersampling to reduce class imbalance

This ensures the dataset is clean, consistent, and suitable for model training.

5.3 Feature Engineering

Feature engineering is a critical step in the DDoS attack detection system, as it transforms raw network traffic data into meaningful representations suitable for different types of models. Separate feature engineering pipelines are designed for traditional machine learning (ML) models and deep learning (DL) models.

Feature Engineering for Machine Learning Models (XGBoost)

For traditional machine learning models such as XGBoost, feature engineering plays a decisive role because these models rely heavily on explicit, well-defined input features. In the proposed system, a compact and carefully selected set of 10 network flow features is used. These features are chosen based on their ability to capture traffic intensity, flow behaviour, and temporal characteristic**s**, which are strong indicators of DDoS attacks.

Features for XGBoost:

1. **Flow Duration**
   Represents the total time duration of a network flow. DDoS attacks often generate either extremely short-lived or unusually long flows compared to normal traffic, making this feature useful for identifying abnormal behaviour.

2. **Total Fwd Packets**
   Indicates the number of packets sent from source to destination. In DDoS scenarios, attackers typically generate a high volume of forward packets toward the target server.

3. **Total Backward Packets**
   Measures the number of packets sent from destination to source. A disproportionally low number of backward packets compared to forward packets is a common symptom of flooding-based DDoS attacks.

4. **Total Length of Fwd Packets**
   Captures the total size of forward packets in bytes. This feature reflects bandwidth consumption and helps differentiate high-volume attack traffic from normal communication.

5. **Fwd Packet Length Max**
   Represents the maximum packet size in the forward direction. Abnormal packet size patterns often appear in attack traffic due to automated packet generation.

6. **Flow Bytes/s**
   Measures the rate of data transfer in bytes per second. DDoS attacks typically exhibit extremely high data rates, making this a key discriminative feature.

7. **Flow Packets/s**
   Indicates the number of packets transmitted per second. This feature directly reflects traffic burstiness and is highly effective for identifying flooding attacks.

8. **Idle Mean**
   Represents the average idle time between packet bursts within a flow. Normal traffic usually exhibits higher variability in idle periods, whereas DDoS traffic tends to have very low or highly uniform idle times.

9. **Active Mean**
   Measures the average active time during which packets are continuously transmitted. Sustained high active times are indicative of attack traffic attempting to overwhelm network resources.

10. **Protocol**
    Identifies the transport-layer protocol (e.g., TCP, UDP, ICMP). Certain protocols are more frequently exploited in DDoS attacks, and this feature helps the model learn protocol-specific attack patterns.

These ten features are deliberately selected to provide maximum discriminatory power with minimal dimensionality. XGBoost is particularly well suited to this approach because it can effectively model non-linear relationships and feature interactions, even with a limited number of well-engineered features. Using only ten features also reduces training time, lowers overfitting risk, and improves real-time inference performance.

Feature Engineering for Deep Learning Models

In contrast to traditional ML models, deep learning models are capable of learning complex representations directly from data. As a result, the feature engineering pipeline for DL models is less restrictive and allows for a broader set of features or raw statistical representations.

For DL-based detection:

- A larger number of flow-level or time-windowed features may be used.
- Features are often normalised or standardised to stabilise gradient-based learning.
- Temporal aggregation (such as sliding windows) can be applied to capture traffic evolution over time.
- The model automatically learns hierarchical and non-linear feature representations, reducing dependence on manual feature selection.

This makes DL models particularly effective for detecting low-rate, stealthy, or evolving DDoS attacks that are difficult to characterise using fixed thresholds.

5.4 Model Design and Training

Machine Learning using XGBoost

The machine learning module of the proposed DDoS detection system is designed using the XGBoost (Extreme Gradient Boosting) algorithm due to its high efficiency and strong performance on structured network traffic data. XGBoost is an ensemble learning technique that combines multiple weak decision tree classifiers to form a strong predictive model. Each successive tree is trained to correct the errors made by previous trees, thereby improving overall accuracy. This iterative boosting mechanism makes XGBoost particularly effective in handling complex patterns in network traffic.

In this project, a set of carefully selected network traffic features is used as input to the XGBoost model. These features include packet count, packet rate, flow duration, packet length statistics, protocol type, and port information, which are known to be significant indicators of DDoS attacks. Prior to model training, the dataset undergoes preprocessing steps such as removal of irrelevant attributes, handling of missing values, and feature scaling using standard normalization techniques. Feature scaling ensures that all input features contribute proportionately to the learning process.

The preprocessed dataset is divided into training and testing subsets to evaluate the model's generalization capability. During training, important hyperparameters such as learning rate, maximum tree depth, and number of estimators are tuned to achieve optimal performance while avoiding overfitting. The objective function used by XGBoost minimizes classification error by applying gradient descent optimization. The trained model is evaluated using metrics such as accuracy, precision, recall, and F1-score to measure its effectiveness in distinguishing between normal and attack traffic.

Pseudo code: XGBoost for DDoS Attack Detection

BEGIN

Step 1: Load the network traffic dataset

Step 2: Perform data preprocessing

Step 3: Split the dataset into training data and testing data

Step 4: Initialize XGBoost parameters

1. Learning rate
2. Maximum tree depth
3. Number of boosting iterations

Step 5: Initialize the model with a base prediction

Step 6: For each boosting iteration

1. Calculate prediction errors (residuals)
2. Compute gradients of the loss function
3. Train a decision tree using gradients
4. Add the new tree to the existing model

Step 7: Repeat Step 6 until all trees are trained

Step 8: Evaluate the trained model using test data

Step 9: Calculate performance metrics

Step 10: Save the trained XGBoost model and scaler

Step 11: Use the saved model to classify live network traffic
as Normal or DDoS Attack

END

Once satisfactory performance is achieved, the trained XGBoost model and the associated feature scaler are saved using Joblib. These saved artifacts are later loaded during the live traffic prediction phase, ensuring consistency between training and inference. The XGBoost model provides fast prediction speed and low computational overhead, making it suitable for real-time DDoS detection scenarios.

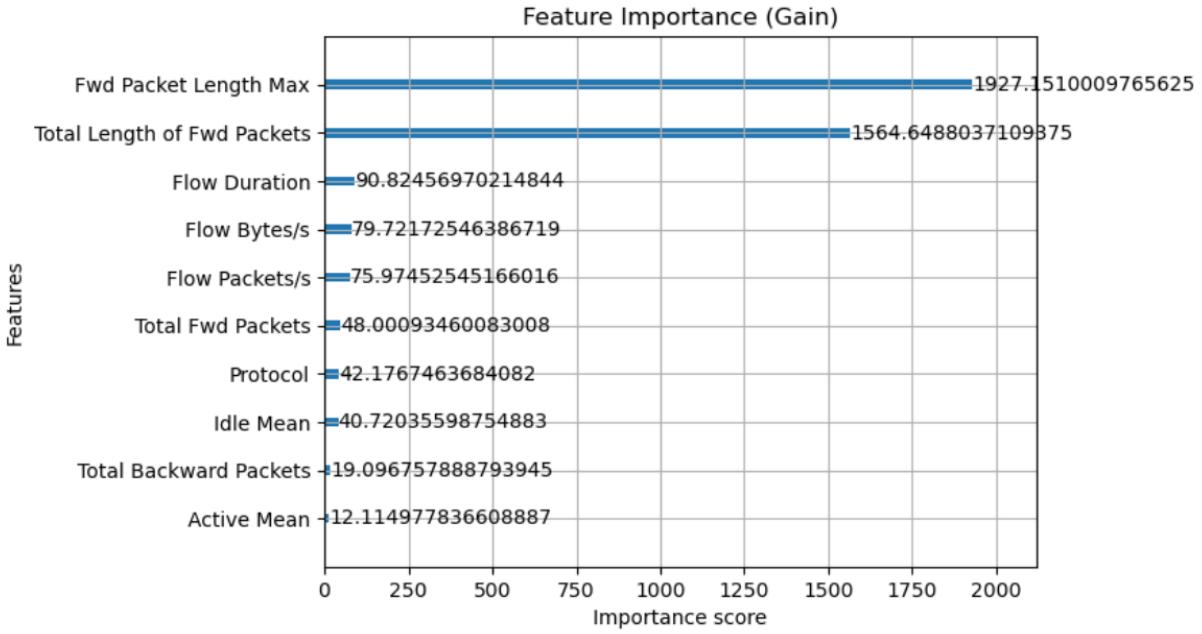


Fig 5.4.1 Feature Importance for XGBoost

Deep Learning using Deep Neural Network (DNN)

The deep learning component of the proposed system employs a Deep Neural Network (DNN) to identify complex and non-linear patterns in network traffic that may indicate sophisticated DDoS attacks. Unlike traditional machine learning models, DNNs are capable of learning hierarchical feature representations, making them well-suited for detecting subtle and evolving attack behaviors.

The DNN architecture designed in this project consists of an input layer, multiple hidden layers, and an output layer. The input layer receives normalized network traffic features identical to those used in the machine learning module, ensuring uniformity across the detection framework. The hidden layers are fully connected and utilize the Rectified Linear Unit (ReLU) activation function, which enables efficient learning and faster convergence. To prevent overfitting, dropout layers are introduced between hidden layers by randomly disabling a fraction of neurons during training.

The output layer uses a sigmoid activation function to perform binary classification, producing a probability score indicating whether the observed traffic is normal or malicious. The model is trained using the binary cross-entropy loss function, which is suitable for binary classification

tasks. The Adam optimizer is employed to adaptively adjust learning rates and accelerate convergence. Early stopping is applied during training to monitor validation loss and prevent overtraining.

The training dataset is split into training and validation sets to assess model performance during learning. After training, the DNN model achieves improved detection accuracy for complex DDoS patterns compared to traditional classifiers. The trained model is saved in serialized format and loaded during live traffic analysis using real-time packet capture modules. This ensures that the model operates efficiently during deployment without retraining.

Pseudo code: Deep Neural Network for DDoS Attack Detection

BEGIN

Step 1: Load the network traffic dataset

Step 2: Perform data preprocessing

Step 3: Split the dataset into training data and testing data

Step 4: Initialize Deep Neural Network

1. Input Layer
2. One or more hidden layers
3. Output Layer
4. Initialize weights and biases

Step 5: For each training epoch

1. Perform forward propagation
2. Compute predicted output
3. Calculate loss using binary cross entropy
4. Perform backpropagation
5. Update weights using Adam optimizer

Step 6: Repeat Step 5 until convergence or early stopping

Step 8: Evaluate the trained model using test data

Step 9: Calculate performance metrics

Step 10: Save the trained DNN model and scaler

Step 11: Use the saved model to classify live network traffic
as Normal or DDoS Attack

END

CHAPTER 6
VALIDATION AND RESULTS

6.1 Model Evaluation

Model evaluation is a critical phase in the proposed DDoS attack detection system, as it determines the effectiveness and reliability of both machine learning and deep learning models in distinguishing normal network traffic from attack traffic. The evaluation process is carried out after model training to assess how well the models generalize to unseen data. In this project, both the XGBoost-based machine learning model and the Deep Neural Network (DNN) model are evaluated using standard performance metrics commonly used in binary classification problems.

The dataset is divided into training and testing sets to ensure unbiased evaluation. The trained models are tested on the unseen test dataset, and predictions are compared with actual labels to measure classification performance. Key evaluation metrics include accuracy, precision, recall, and F1-score. Accuracy measures the overall correctness of predictions, while precision indicates the proportion of correctly identified attack instances among all predicted attacks. Recall measures the ability of the model to correctly detect actual attack traffic, which is especially important in security applications. The F1-score provides a balanced measure by combining precision and recall.

In addition to these metrics, confusion matrices are used to analyze the classification results in terms of true positives, true negatives, false positives, and false negatives. This analysis helps in understanding the types of errors made by the model and their potential impact on real-time detection. A low false negative rate is particularly desirable, as undetected DDoS attacks can severely affect network availability.

For the deep learning model, validation loss and validation accuracy are monitored during training to prevent overfitting. Early stopping techniques are applied when the validation performance stops improving. The machine learning model is evaluated for both accuracy and prediction speed, as fast inference is essential for live traffic monitoring. The evaluation results indicate that the

DNN model achieves higher accuracy for complex attack patterns, while the XGBoost model provides faster and more efficient predictions.

Overall, the evaluation results demonstrate that the proposed system achieves reliable and accurate DDoS detection. The combination of quantitative metrics and error analysis confirms the suitability of the models for real-time deployment in network security environments.

Model performance is evaluated using standard classification metrics:

- Accuracy
- Precision
- Recall
- F1-Score

Comparative analysis is performed between machine learning and deep learning models to select the most robust model.

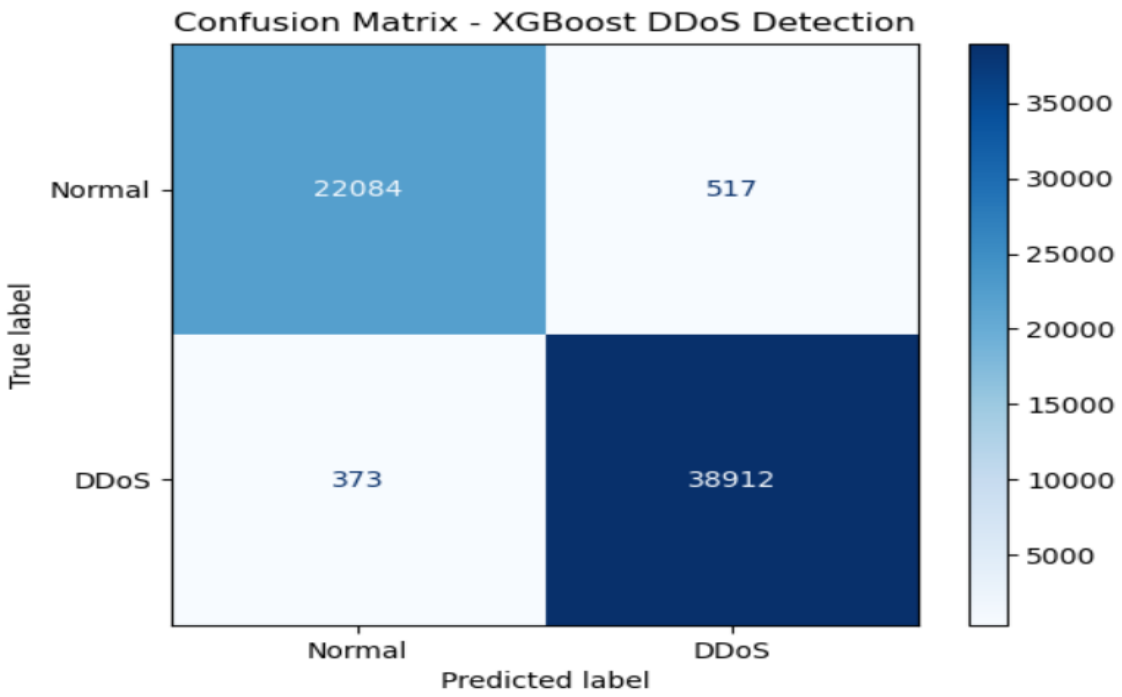


Fig 6.1.1 Confusion matrix for machine learning model

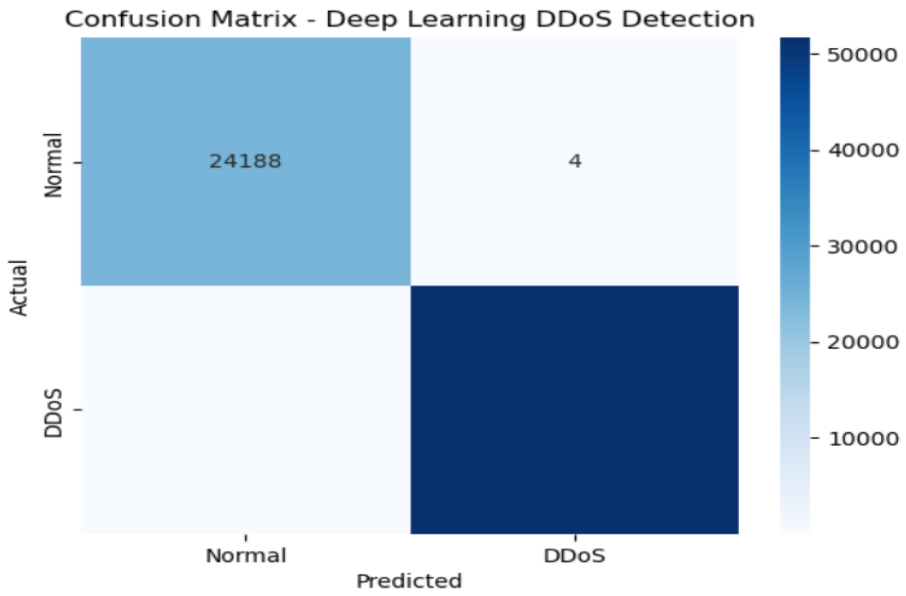


Fig 6.1.2 Confusion matrix for deep learning model

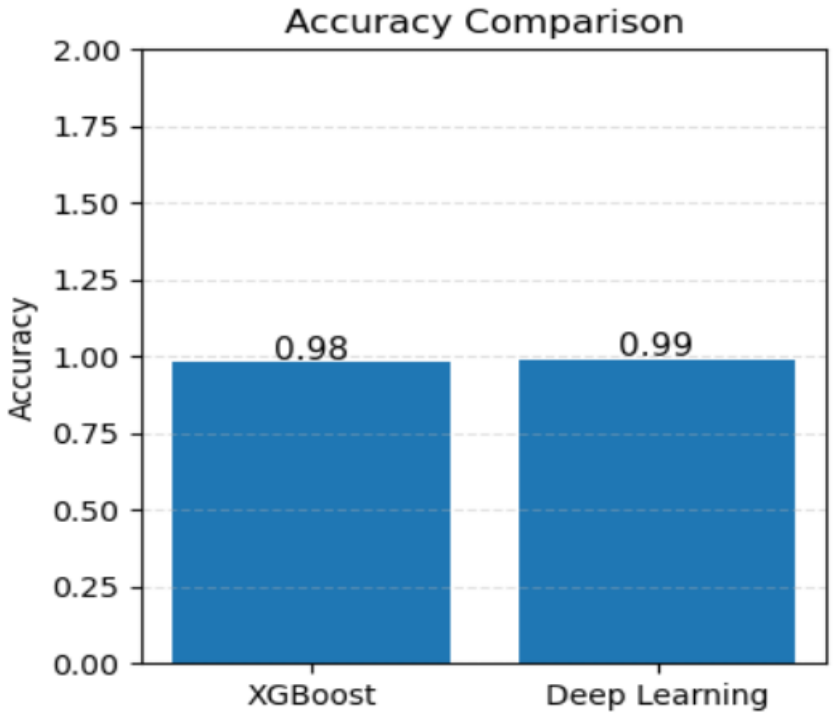


Fig 6.1.3 Accuracy comparison

6.2 Dashboard-Based Validation

Dashboard-based validation plays a crucial role in evaluating the performance and usability of the proposed DDoS attack detection system. The dashboard acts as an interactive interface that allows users to visualize model predictions, analyze performance metrics, and compare the effectiveness of machine learning and deep learning models. This form of validation helps in understanding system behavior in a more intuitive and practical manner, especially during offline testing scenarios.

6.2.1 Dashboard Implementation Overview

The dashboard is implemented to provide a user-friendly interface for monitoring and validating the DDoS detection system. It presents the outputs of trained models in a visual format, making it easier for users to interpret results without analyzing raw numerical data. The dashboard is designed to display both machine learning (XGBoost) and deep learning (DNN) model outputs.

The interface includes sections for displaying prediction status.The dashboard also shows classification results indicating whether the analyzed traffic is normal or a DDoS attack. This visualization improves system transparency and supports better decision-making. The dashboard is designed with a user-friendly interface, ensuring easy navigation and clear presentation of information. The layout is structured to minimize complexity while providing all necessary insights required for validation and analysis. This improves system usability, especially for users with limited technical expertise.

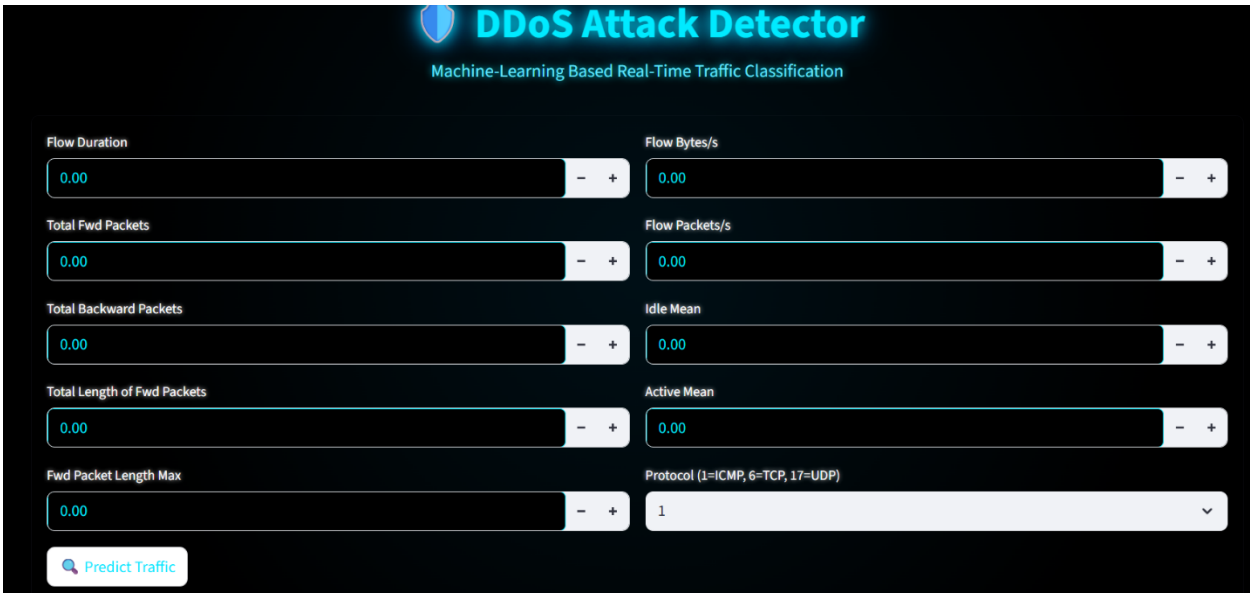


Fig 6.2.1 Dashboard Interface

6.2.2 Offline Testing Through Dashboard

Offline testing through the dashboard is performed using pre-recorded datasets containing both normal and DDoS attack traffic. The trained XGBoost and DNN models are loaded into the dashboard, and test data is passed through the system to generate predictions. The output of each model is visualized in real time on the dashboard.

Offline Testing Using XGBoost Model (User Input Based)

Offline testing of the XGBoost model is performed using manually entered user input values. In this approach, the user provides network traffic feature values such as packet rate, flow duration, packet size statistics, protocol type, and port information through an input interface. These values simulate network traffic conditions and allow testing of the model without relying on live traffic capture or dataset files.

Once the user inputs the feature values, the data is preprocessed and scaled using the same preprocessing steps applied during model training. The processed input is then passed to the trained XGBoost model, which classifies the traffic as either *Normal* or *DDoS Attack*. The prediction result is immediately displayed on the dashboard or console interface.

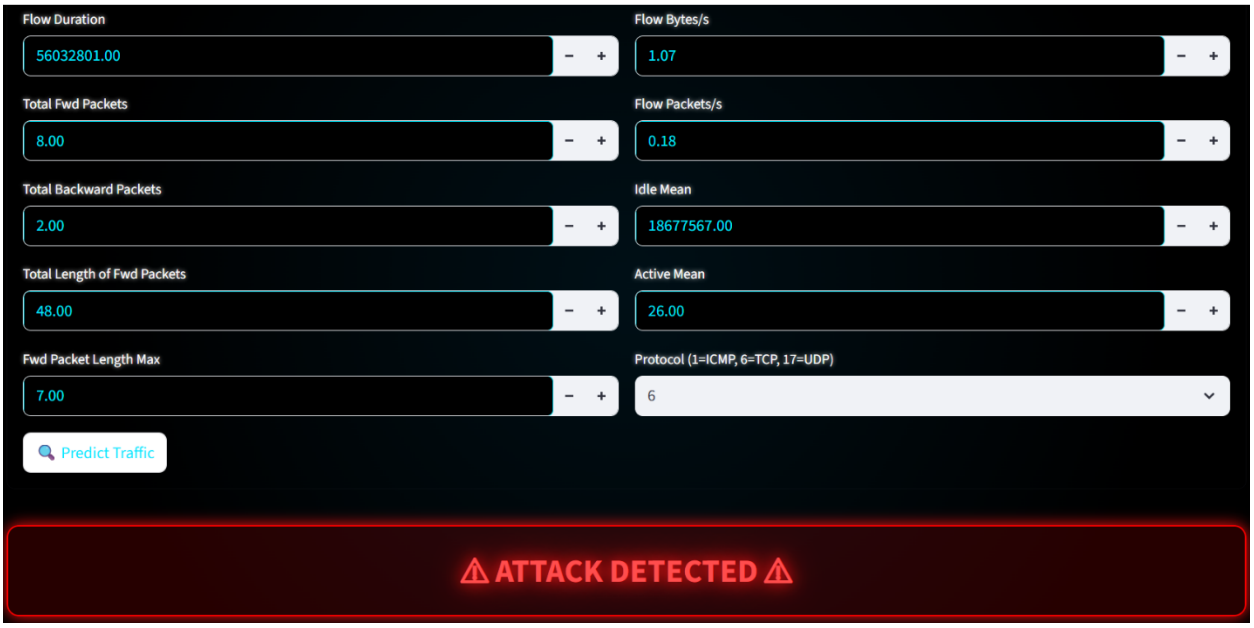


Fig 6.2.2 Offline Testing Results Displayed on Dashboard for XGBoost

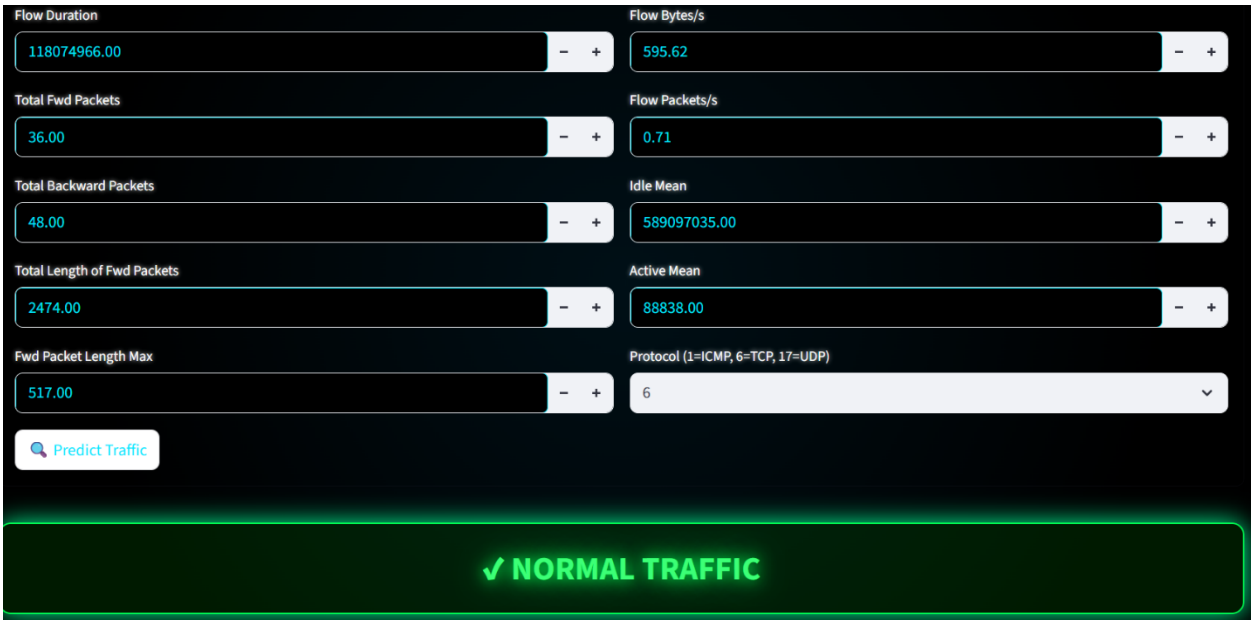
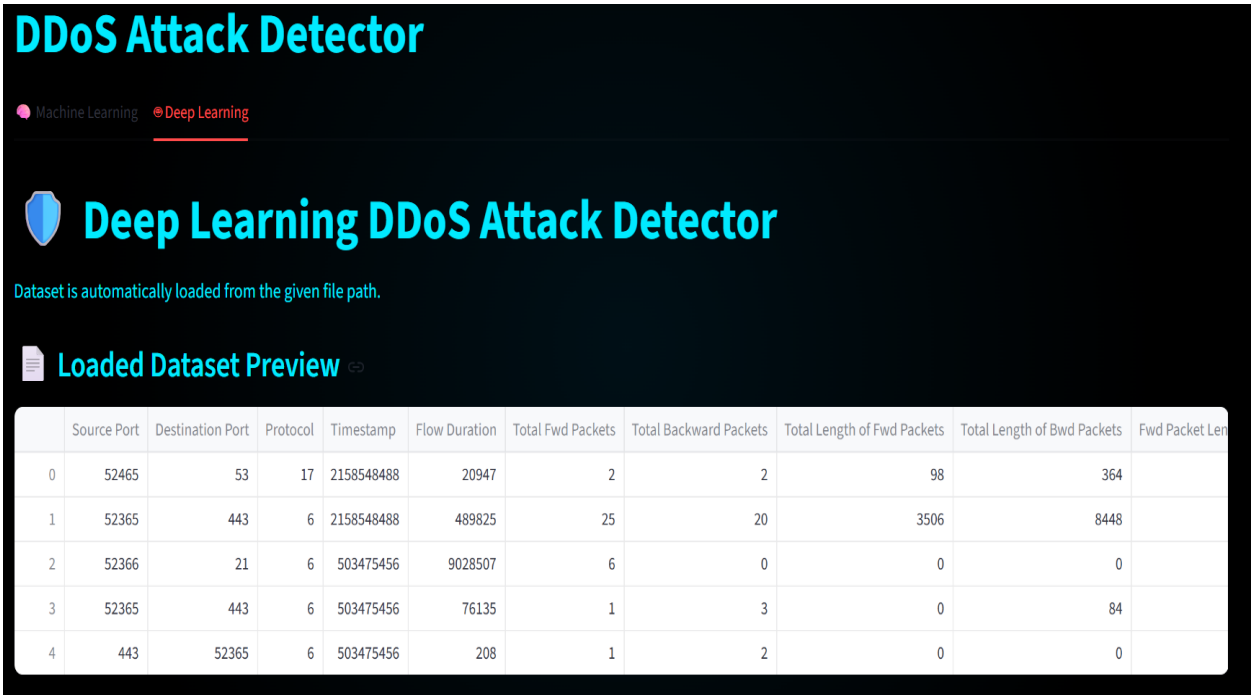


Fig 6.2.3 Offline Testing Results Displayed on Dashboard for XGBoost

Offline Testing Using DNN Model (CSV File Based)

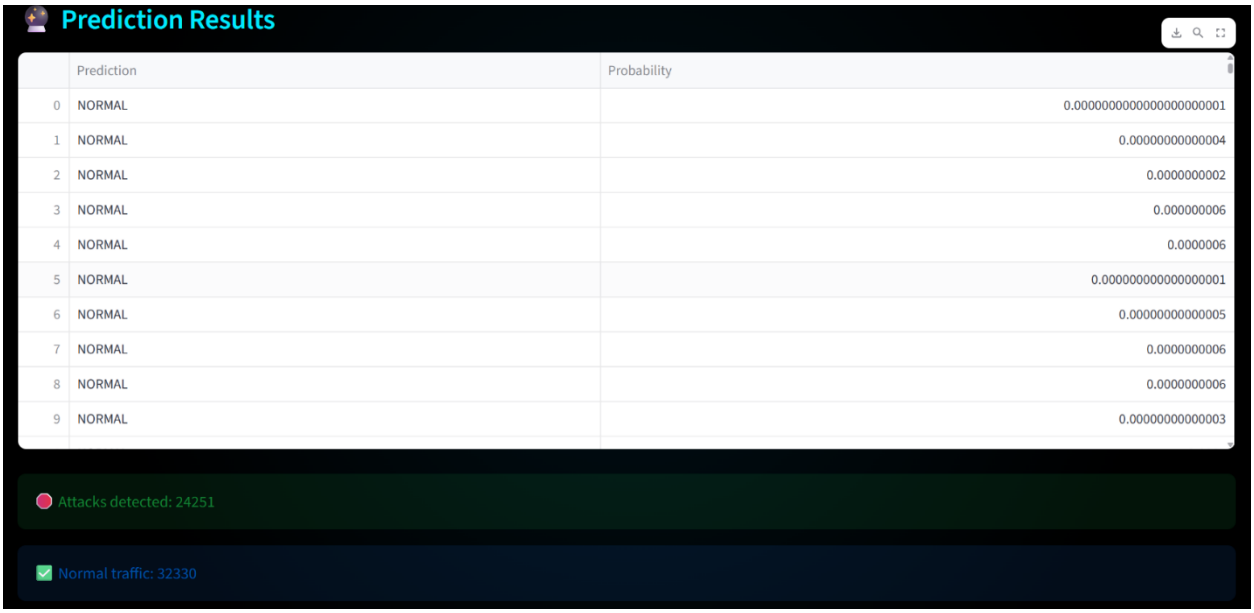
Offline testing of the Deep Neural Network (DNN) model is carried out using a CSV file containing multiple network traffic records. The CSV file consists of pre-extracted and labeled network traffic features similar to those used during training. This approach allows batch testing of the DNN model on multiple samples at once, making it suitable for evaluating overall model performance.

During testing, the CSV file is uploaded or loaded into the system, and the data is preprocessed and normalized using the saved feature scaler. Each record in the CSV file is passed through the trained DNN model to generate predictions. The output includes classification results indicating whether each traffic instance is normal or represents a DDoS attack.



	Source Port	Destination Port	Protocol	Timestamp	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Len
0	52465	53	17	2158548488	20947	2	2	98	364	
1	52365	443	6	2158548488	489825	25	20	3506	8448	
2	52366	21	6	503475456	9028507	6	0	0	0	
3	52365	443	6	503475456	76135	1	3	0	84	
4	443	52365	6	503475456	208	1	2	0	0	

Fig 6.2.4 Offline Testing Results Displayed on Dashboard for DNN



Prediction Results

	Prediction	Probability
0	NORMAL	0.00000000000000000000001
1	NORMAL	0.00000000000004
2	NORMAL	0.0000000002
3	NORMAL	0.000000006
4	NORMAL	0.0000006
5	NORMAL	0.000000000000000001
6	NORMAL	0.00000000000005
7	NORMAL	0.0000000006
8	NORMAL	0.0000000006
9	NORMAL	0.00000000000003

Attacks detected: 24251

Normal traffic: 32330

Fig 6.2.5 Offline Testing Results Displayed on Dashboard for DNN

6.1.3 Dashboard Result Analysis

The results observed through the dashboard provide valuable insights into model behavior and system performance. Visualization of performance metrics shows that both XGBoost and DNN models are capable of accurately detecting DDoS attacks. The machine learning model demonstrates faster prediction speed and stable performance on structured traffic data, while the deep learning model exhibits higher accuracy in identifying complex and high-volume attack patterns.

The comparative graphs help in identifying strengths and limitations of each model. The dashboard also allows observation of model consistency across different test samples. Any misclassification or fluctuation in predictions can be easily detected through visual analysis. This helps in fine-tuning model parameters and improving overall system robustness.

Overall, dashboard-based validation confirms that the proposed system performs effectively under offline testing conditions and provides clear, interpretable results. The visualization-driven approach enhances system reliability and supports practical deployment of the DDoS detection framework.

6.3 Live Traffic Capture Testing

Live traffic capture testing is performed to validate the real-time detection capability of the proposed DDoS attack detection system. Unlike offline testing, this phase evaluates the system under actual network conditions by capturing and analyzing live network packets. The objective of this testing is to ensure that the system can accurately detect normal traffic with minimal delay during real-time operation since live DDoS simulation is unethical.

6.3.1 Live Traffic Capture Setup

The live traffic capture setup involves monitoring real-time network packets through the system's network interface. The detection system is configured to continuously capture incoming and outgoing network traffic from the active network interface such as Ethernet or Wi-Fi. This setup enables the system to observe real network behavior instead of relying solely on pre-recorded datasets.

Captured packets are processed in real time, and relevant network features are extracted from each packet or traffic flow. These features include packet count, packet rate, flow duration, packet size statistics, protocol type, and port numbers. Feature extraction is performed using the same methodology adopted during the training phase to maintain consistency between training and live testing. The extracted features are then normalized using the previously saved feature scaler before being passed to the trained machine learning and deep learning models.

This setup ensures that the system is capable of handling continuous traffic streams and prepares the data in a format suitable for real-time prediction.

6.3.2 Real-Time Prediction Results

During live traffic capture testing, the extracted features are passed to the trained XGBoost and Deep Neural Network (DNN) models for real-time prediction. The models classify the incoming traffic as either *Normal* or *DDoS Attack* based on learned traffic patterns.

Under normal network conditions, the system successfully identifies legitimate traffic and classifies it as normal. This demonstrates that the detection system does not generate false alerts during regular usage.

The real-time prediction results are displayed through the PowerShell providing immediate feedback to the user. This confirms that the system can perform live detection with minimal latency and accurate classification.

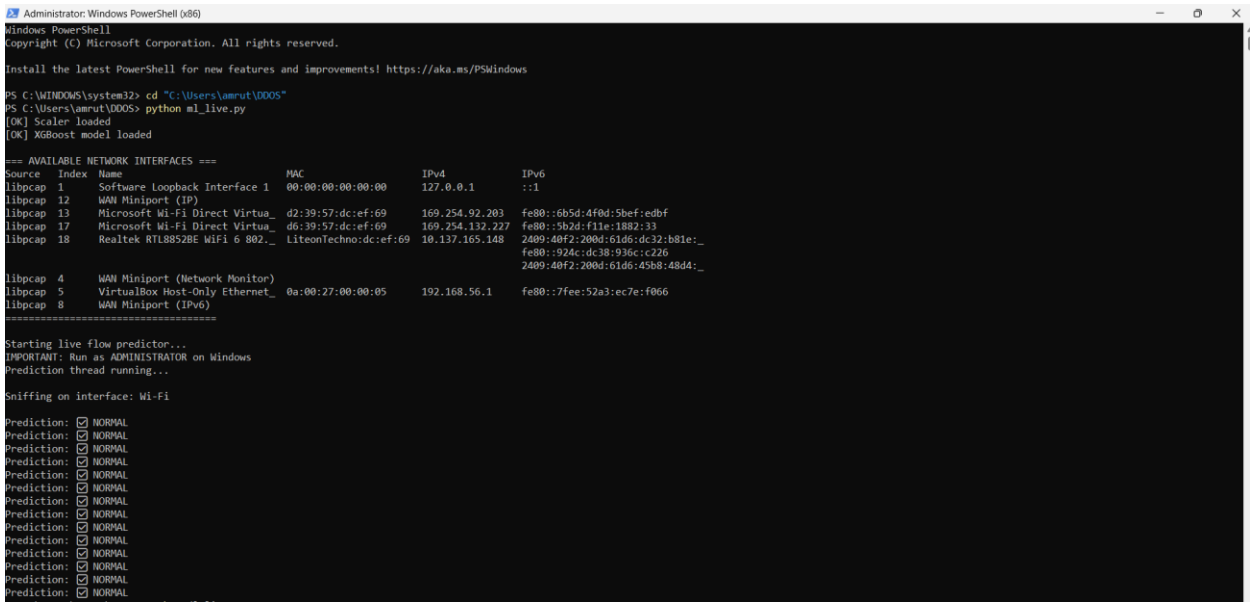


Fig 6.3.1 Real-Time Normal Traffic Detection Output using XGBoost model

```
PS C:\Users\amrut\DDOS> python dl_live3.py
2026-01-02 12:14:02.790598: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation
 orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-01-02 12:14:07.479408: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation
 orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
[OK] Scaler loaded
2026-01-02 12:14:11.276436: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
[OK] DNN model loaded
[OK] Feature count: 70

=== AVAILABLE NETWORK INTERFACES ===
Source   Index  Name                                          MAC                  IPv4            IPv6
libpcap  1      Software Loopback Interface 1                 00:00:00:00:00:00    127.0.0.1       ::1
libpcap  12     WAN Miniport (IP)
libpcap  13     Microsoft Wi-Fi Direct Virtual Adapter        d2:39:57:dc:ef:69    169.254.92.203  fe80::6b5d:4f0d:5bef:edbf
libpcap  17     Microsoft Wi-Fi Direct Virtual Adapter #2     d6:39:57:dc:ef:69    169.254.132.227 fe80::5b2d:f11e:1882:33
libpcap  18     Realtek RTL8852BE WiFi 6 802.11ax PCIe Adapter LiteonTechno:dc:ef:69 10.137.165.148  2409:40f2:200d:61d6:45b8:48d4:3da:98c5
                                                                                                     2409:40f2:200d:61d6:dc32:b81e:36e6:2990
                                                                                                     fe80::924c:dc38:936c:c226

libpcap  4      WAN Miniport (Network Monitor)
libpcap  5      VirtualBox Host-Only Ethernet Adapter         0a:00:27:00:00:05    192.168.56.1    fe80::7fee:52a3:ec7e:f066
libpcap  8      WAN Miniport (IPv6)
==================================

Starting LIVE DNN DDoS Detector (70-feature mode)
Run as ADMINISTRATOR
DNN live prediction thread started...

Sniffing on interface: Wi-Fi

Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
Prediction : ☑ NORMAL
```

Fig 6.3.2 Real-Time DDoS Attack Detection Output using Deep Neural Network model

6.2.3 Analysis of Live Detection Results

The analysis of live detection results focuses on evaluating the system's detection speed and accuracy during real-time monitoring. The system demonstrates fast response time, as predictions are generated immediately after feature extraction without noticeable delay. This low latency is essential for real-world DDoS detection, where timely identification of attacks is critical.

6.4 Local Testing and Trigger-Based Validation

Local testing and trigger-based validation are performed to evaluate the behavior of the proposed DDoS attack detection system at the system level under controlled conditions. This phase focuses on validating the correctness, stability, and efficiency of the system when monitoring localhost or internal traffic and when enabling or disabling detection using a trigger mechanism. Such testing ensures that the system functions reliably before and during real-time deployment.

6.4.1 Local Traffic Testing

Local traffic testing is carried out by monitoring traffic generated within the local system or internal network environment. In this testing scenario, the detection system captures localhost or internal network traffic without external attack sources. The primary objective of local testing is to observe system behavior under normal operating conditions and ensure that the detection models do not generate false alerts.

During local testing, network traffic generated by regular system activities such as browsing, file access, and background services is captured. Features such as packet rate, flow duration, and packet size statistics are extracted from this traffic and passed to the trained XGBoost and DNN models. Since the traffic corresponds to normal system usage, the models correctly classify it as normal traffic.

Local testing confirms that the system remains stable, responsive, and accurate when deployed on the host machine. It also verifies proper integration of packet capture, feature extraction, preprocessing, and prediction modules under non-attack conditions.

6.4.2 Trigger-Based Testing

Trigger-based testing is implemented to validate the system's ability to start and stop real-time detection based on user control. The trigger mechanism allows the user or

administrator to enable or disable live traffic monitoring as required. This feature improves system efficiency by preventing unnecessary computation when detection is not needed.

In this testing phase, the trigger is manually activated to start traffic monitoring and prediction. When the trigger is turned ON, the system begins capturing traffic, extracting features, and classifying traffic using trained XGBoost and DNN models. When the trigger is turned OFF, packet capture and prediction processes are halted, and the system returns to an idle state.

Trigger-based testing confirms that the system responds correctly to start and stop commands without causing errors or instability. It also demonstrates improved resource utilization, as CPU and memory usage are reduced during inactive periods. This mechanism is particularly useful in real-world scenarios where continuous monitoring may not always be required.

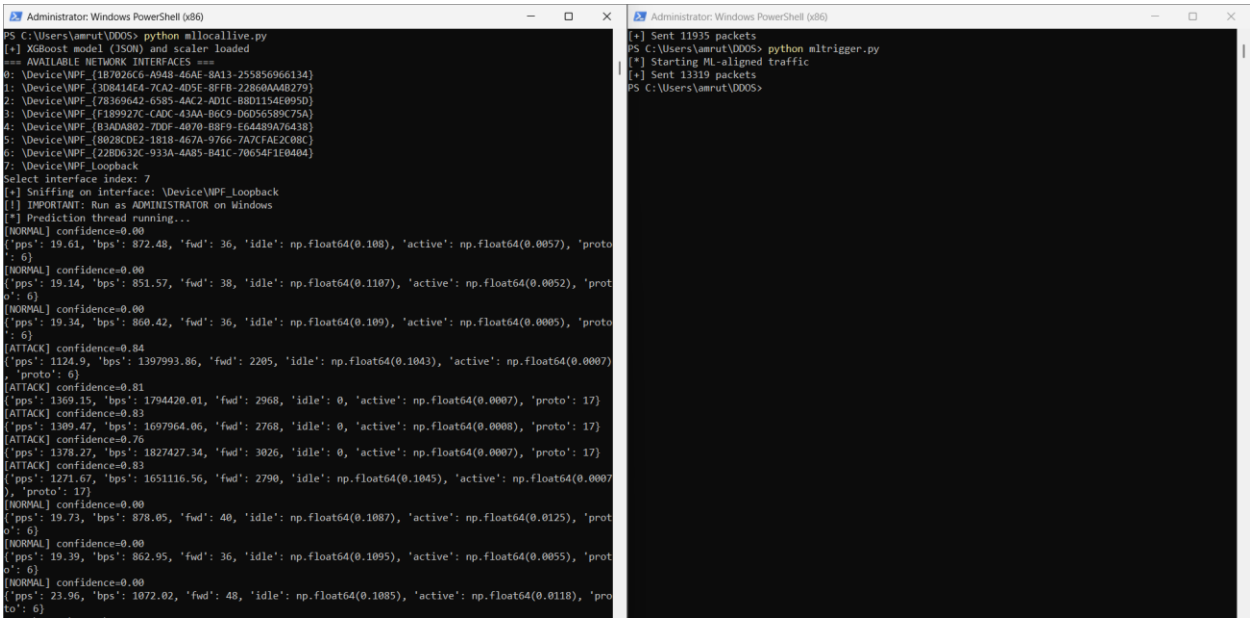


Fig 6.4.1 Trigger-Based Detection Control for XGBoost Model

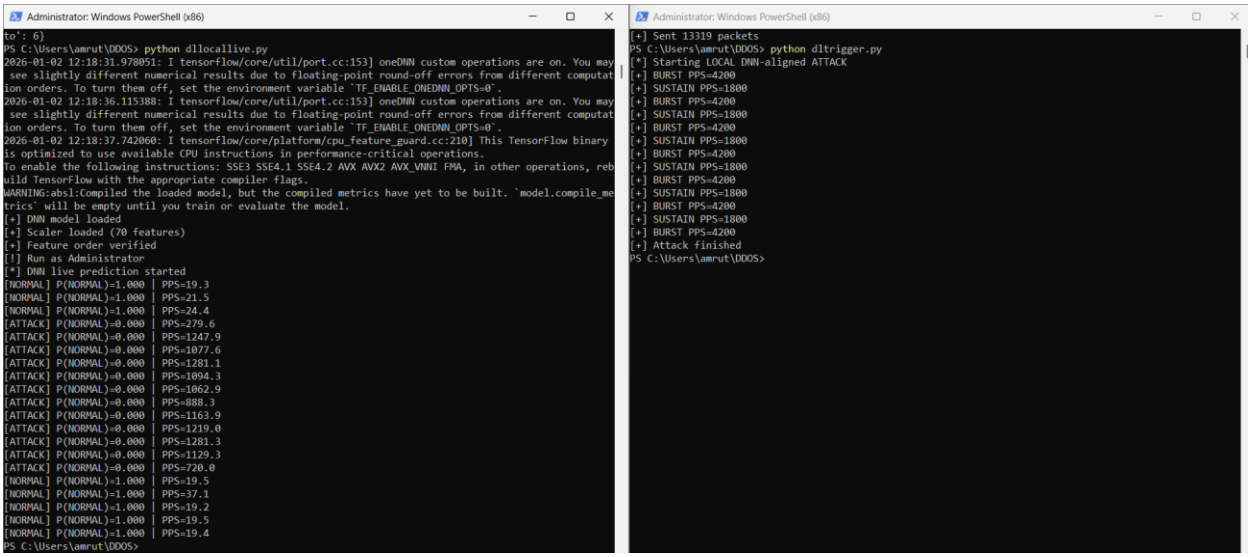


Fig 6.4.2 Trigger-Based Detection Control for DNN Model

Analysis of Local and Trigger-Based Testing

The results obtained from local and trigger-based testing indicate that the proposed system functions correctly under system-level conditions. Both XGBoost and DNN models accurately classify normal local traffic without false positives. The trigger mechanism effectively controls detection execution and improves overall system efficiency. These tests confirm that the system is robust, stable, and suitable for deployment in real-time network environments.

CONCLUSION

In this project, an intelligent and efficient system for detecting Distributed Denial of Service (DDoS) attacks using machine learning and deep learning techniques has been successfully designed and implemented. The proposed system focuses on analyzing network traffic patterns and classifying them as either normal or attack traffic in both offline and real-time environments. By leveraging data-driven approaches, the system overcomes the limitations of traditional rule-based intrusion detection mechanisms.

The project implements a hybrid detection framework using a machine learning model based on XGBoost and a deep learning model based on a Deep Neural Network (DNN). The XGBoost model provides fast and efficient predictions using structured network features, while the DNN model effectively captures complex and non-linear traffic patterns associated with sophisticated DDoS attacks. Offline testing, dashboard-based validation, live traffic capture testing, and local system testing confirm the reliability and accuracy of the proposed system.

The inclusion of a dashboard enhances system usability by providing clear visualization of prediction results and performance metrics such as accuracy, precision, recall, and F1-score. The trigger-based mechanism further improves system efficiency by allowing controlled execution of real-time detection. Experimental results demonstrate that the proposed system accurately detects DDoS attacks with minimal latency and maintains stable performance under different testing scenarios.

Overall, the project successfully achieves its objectives by delivering a scalable, accurate, and real-time DDoS attack detection system suitable for practical network security applications.

Future Scope

Although the proposed system performs effectively, there is significant scope for further enhancement and expansion. One possible future improvement is the integration of additional deep learning architectures such as Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) to further improve detection accuracy for evolving and complex attack patterns. Ensemble techniques combining multiple models can also be explored to enhance robustness.

The system can be extended to detect other types of network attacks such as port scanning, brute-force attacks, botnet activities, and intrusion attempts. Incorporating automated mitigation mechanisms, such as traffic filtering or rate limiting, can enable the system to not only detect but also respond to attacks in real time.

Another important area for future work is deploying the system in cloud or distributed environments to handle large-scale network traffic. Integration with Software Defined Networking (SDN) can allow dynamic traffic control and centralized monitoring. Additionally, incorporating online learning techniques can enable the system to continuously adapt to new attack patterns without retraining from scratch.

Enhancing the dashboard with advanced analytics, alert notifications, and historical traffic analysis can further improve usability. With these enhancements, the proposed system can evolve into a comprehensive and intelligent network security solution capable of protecting modern network infrastructures.

REFERENCES

[1] Mohammad Najafimehr, Sajjad Zarifzadeh, and Seyed Akbar Mostafavi, "DDoS attacks and machine-learning-based detection methods: A survey and taxonomy," Engineering Reports, May 30, 2023.

[2] Raihan Putra Janivasya and Ika Dyah Agustia Rachmawati, "DDoS Detection using Machine Learning Approach," Procedia Computer Science, 2024.

[3] Mahmood A. Al-Shareeda, Selvakumar Manickam, Murtaja Ali Saare, "DDoS attacks detection using machine learning and deep learning: analysis and comparison", 2023.

[4] Meenakshi Mittal, Krishan Kumar, and Sunny Behal, "Deep learning approaches for detecting DDoS attacks: a systematic review," Artif. Intell, 2022.

[5] Sheeraz Ahmed, Zahoor Ali Khan, Syed M. Mohsin, Shahid Latif, Sheraz Aslam, Hana Mujlid, M. Adil, and Zeeshan Najam, "Effective and Efficient DDoS Attack Detection Using Deep Learning Algorithm, Multi-Layer Perceptron," Future Internet, 2023.

[6] S. Abhiramsundari and V. Ramaswamy, "Distributed denial-of-service (DDOS) attack detection using supervised learning algorithms", Scientific reports, 2025.

[7] Xingbing Fu and Supeng Lou, "Deep Learning techniques for DDoS attack detection",2025.

[8] Sharmin Aktar and Abdullah Yasin Nur," Towards DDoS attack detection using deep learning approach",2023.

[9] Haya Malooh Alqahtani and Monir Abdullah, "A Review on DDoS Attacks Classifying and Detection by ML/DL Models," International Journal of Advanced Computer Science and Applications (IJACSA), 2024.

[10] Firooz B. Saghezchi, Georgios Mantas, Manuel A. Violas, "Machine Learning for DDoS Attack Detection in Industry 4.0 CPPSs", 2022.

[11] Cloudflare, "DDoS threat report for Q2 2025," Cloudflare Blog, Jul. 2025.

[Online]. Available: https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/

[12] A. A. Diro and N. Chilamkurti, “Distributed attack detection scheme using deep learning approach for Internet of Things,” Future Generation Computer Systems, vol. 82, pp. 761–768, May 2018.

[13] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” Proc. ICISSP, pp. 108–116, 2018.